**Cloud Programming and Operations 2025-26**

# Progetto Capstone: EuroTransit Marketplace

*Costruire, distribuire, osservare e irrobustire un sistema cloud-native resiliente*

## Introduzione

Questo è il progetto finale e integrativo del corso. Tutto ciò che avete messo in pratica dal Lab01 al Lab05 (build/test/CI, containerizzazione, workload Kubernetes, ingress e TLS, operatori, delivery GitOps e osservabilità) converge ora su un unico sistema che dovete progettare per il fallimento, distribuire tramite Git, osservare sotto carico e dimostrare resiliente in una presentazione dal vivo.

I lab vi hanno insegnato a far funzionare un sistema e a renderlo comprensibile. Il capstone pone la domanda più difficile verso cui il corso ha lavorato:

> *La stabilità non è una proprietà. È un risultato. Riuscite a produrre quel risultato di proposito, in condizioni di fallimento parziale, e a dimostrarlo?*

Costruirete **EuroTransit**, un marketplace multi-servizio piccolo ma realistico. Il punto non è la ricchezza di funzionalità. Il punto è il comportamento operativo: come il sistema tiene insieme quando un servizio rallenta, quando un Pod muore a metà di una richiesta, quando il database fa failover, quando una partizione di rete isola un componente, e quando distribuite una nuova versione in mezzo a tutto questo senza mandare giù il sistema.

Questo progetto è anche il punto in cui il coding agentico è esplicitamente permesso e atteso, alle condizioni descritte di seguito. Sarete valutati molto più sui giudizi che solo il vostro team può prendere che sugli artefatti che un agente può generare.

## Obiettivi di apprendimento

Al termine di questo progetto, il vostro team dovrebbe essere in grado di:

- decomporre un dominio in servizi cooperanti con confini sincroni e asincroni espliciti;
- giustificare un modello di consistenza (CAP / PACELC) per una risorsa contesa e implementarlo;
- implementare una pipeline di elaborazione asincrona usando le coroutine / i Flow di Kotlin, con semantica corretta di cancellazione e arresto;
- rendere sicure le operazioni cross-service in caso di retry, duplicati e fallimento parziale (idempotenza, circuit breaker, bulkhead, backpressure, degradazione con grazia);
- definire SLO significativi e gli SLI, le dashboard e gli alert basati sui sintomi che li proteggono;
- distribuire le modifiche tramite GitOps con almeno due strategie di progressive delivery (canary e blue/green);
- progettare ed eseguire esperimenti di caos controllati e dimostrare che i fallimenti parziali restano contenuti;
- ragionare sulla postura di sicurezza e governance degli attori non umani (CI, controller e agenti di coding) all'interno del vostro loop di delivery;
- gestire un incidente dal vivo durante la presentazione, ripristinare il servizio e produrre un postmortem blameless (senza colpevolizzazioni).

## Il dominio EuroTransit

EuroTransit è un marketplace per la vendita di biglietti del treno (potete reskinnare liberamente il dominio; ciò che conta sono i pattern di interazione). È composto da cinque servizi:

| Servizio | Responsabilità | Stile di interazione |
|---|---|---|
| Catalog | Elenca prodotti/offerte; prevalentemente letture | API sincrona; tollerante alla staleness (dati non aggiornati) |
| Orders | Orchestra il workflow di acquisto | Ingresso sincrono più pipeline asincrona |
| Inventory | Tiene traccia dei posti finiti; la risorsa contesa | Prenotazione sincrona più eventi asincroni |
| Payments | Autorizza il pagamento; non deve addebitare due volte | Chiamata sincrona con stretta idempotenza |
| Notifications | Invia le conferme | Completamente asincrono; il fallimento deve degradare con grazia |

**Forma di interazione richiesta:**

- Un API gateway / ingress pubblico (Traefik, dal Lab03) è l'unico punto di ingresso nord-sud.
- API sincrone tra client → gateway → Catalog/Orders, e Orders → Inventory/Payments dove una decisione deve essere presa adesso.
- Comunicazione asincrona tramite Kafka (distribuito con l'operatore Strimzi) per la pipeline degli ordini e tutto l'eventing (es. `order-placed`, `inventory-reserved`, `payment-authorized`, `order-confirmed`, `notification-requested`).
- Orders possiede un database PostgreSQL gestito dall'operatore CloudNativePG (dal Lab03).
- Notifications deve poter fallire completamente senza far fallire il checkout (degradazione con grazia).

Il “money path” (effettuare un ordine, riservare l'inventario, autorizzare il pagamento, confermare) è il percorso critico. I vostri SLO, le dashboard e gli esperimenti di caos devono essere centrati su di esso.

## Cosa dovete dimostrare (i quattro pilastri)

Il progetto è organizzato attorno a quattro pilastri. Ciascuno è mappato sui moduli del corso e su specifici elementi della Definition-of-Done.

**Pilastro A: Progettazione distribuita ed esecuzione asincrona**

- Una chiara decomposizione dei servizi con confini sincroni vs. asincroni documentati e una giustificazione per ciascuno.
- La pipeline degli ordini implementata in modo asincrono usando le coroutine / i Flow di Kotlin: un ordine effettuato tramite un'API sincrona ritorna rapidamente, mentre prenotazione, pagamento e conferma procedono attraverso fasi guidate da Kafka.
- Comportamento corretto del ciclo di vita asincrono, dimostrato anziché dichiarato:
  - concorrenza strutturata (un CoroutineScope come dominio di fallimento);
  - cancellazione cooperativa su SIGTERM / arresto del Pod (il lavoro in corso termina o viene cancellato in modo pulito, senza task orfani e senza doppia elaborazione);

    - la readiness passa a “rifiuto del traffico” mentre il lavoro in corso si esaurisce (drena).
- Una breve analisi scritta: dove l'asincronia riduce i costi / la pressione di scaling in EuroTransit, e dove invece non aiuterebbe (lavoro CPU-bound)? Fate riferimento al modello blocking-vs-suspending della lezione sull'asincronia.

**Pilastro B: Consistenza in condizioni di contesa (il problema dell'inventario)**

L'inventario è l'unico punto in cui la correttezza confligge realmente con la disponibilità. Due clienti non devono comprare l'ultimo posto.

- Scegliete e giustificate un modello di consistenza per la prenotazione dell'inventario usando i termini CAP / PACELC. Dichiarate esplicitamente cosa sacrificate in una partizione e quale trade-off latenza/consistenza accettate quando non c'è partizione.
- Implementatelo (es. prenotazione condizionale/atomica in PostgreSQL, una macchina a stati di prenotazione, o una strategia di riconciliazione documentata con azioni compensative).
- L'idempotenza è obbligatoria lungo tutto il money path: un evento `order-placed` duplicato o un'autorizzazione di pagamento ritentata non devono né riservare due volte né addebitare due volte. Usate chiavi di idempotenza / deduplicazione e documentate lo schema.
- Dimostrate, sotto un esperimento di caos, che l'invariante (“mai oversell” / mai vendere più del disponibile) regge anche quando i messaggi sono duplicati o un Pod muore a metà prenotazione.

**Pilastro C: Ingegneria della resilienza**

Il sistema deve contenere il fallimento parziale anziché propagarlo.

- Circuit breaker sulle chiamate sincrone cross-service (es. Orders → Payments), con una policy open/half-open definita e un fallback sicuro (in cache, degradato, accodato, o un errore esplicito anziché un blocco illimitato).
- Bulkhead: pool di risorse isolati così che un downstream lento non possa esaurire thread/connessioni condivisi e far cadere flussi non correlati.
- Timeout e retry limitati con backoff più jitter su ogni chiamata remota. Nessuna attesa illimitata; nessuna tempesta di retry ingenua.
- Backpressure / load shedding: in caso di sovraccarico, il sistema rifiuta il lavoro in modo controllato (es. HTTP 429) anziché accodare fino al collasso.
- Degradazione con grazia: ad esempio, Notifications può essere completamente fuori uso e il checkout ha comunque successo.
- Resilienza a livello Kubernetes configurata deliberatamente anziché lasciata di default:
    - probe di startup / readiness / liveness significativi (la liveness non deve fallire per problemi transitori del downstream);
    - PodDisruptionBudget;

**Pilastro D: Delivery, osservabilità e prova sotto fallimento**

- La delivery GitOps prosegue dal Lab04: la CI builda/testa/pubblica immagini immutabili e aggiorna il repository di configurazione; Argo CD riconcilia. La CI non deve detenere credenziali del cluster.
- Due strategie di progressive delivery, entrambe dimostrate:
    - Canary (instradate una piccola frazione del traffico verso una nuova versione, osservate gli SLI, promuovete o abortite; affidatevi a TraefikService per l'implementazione);

    - Blue/green (tirate su la nuova versione, commutate il traffico, mantenete un rollback rapido);
    - Discutete dove le due rimanenti strategie allineate al DORA (all-at-once, rolling) si collocherebbero e perché non le avete usate sul percorso critico.
- L'osservabilità prosegue dal Lab05: dashboard RED per servizio, USE/Golden-Signals per l'infrastruttura, e alert basati sui sintomi legati ai vostri SLO (allarmate sul burn / sui sintomi visibili dall'utente, non sulla CPU).
- SLO per il percorso critico: come minimo uno SLO di latenza e uno SLO di success-rate (tasso di successo) per il checkout, con gli SLI che li misurano e una dichiarazione di error-budget.
- Tracing distribuito lungo il money path (richiesta → gateway → Orders → Inventory/Payments → fasi Kafka), così da poter rispondere a “dove ha speso il suo tempo questo ordine, e dove è fallito?”.

## Chaos engineering

La resilienza che non è mai stata testata è un'ipotesi, non una proprietà. Validerete la vostra.

**Esperimenti di caos controllati (durante lo sviluppo)**

Usando Chaos Mesh (o un equivalente), progettate ed eseguite esperimenti come ipotesi, seguendo il metodo scientifico della lezione sulla resilienza: definite lo stato stazionario (i vostri SLI), enunciate un'ipotesi, iniettate un fallimento, osservate, concludete. Come minimo:

1. Iniezione di latenza in Payments: il circuit breaker di Orders si apre e il fallback si attiva, mentre la navigazione del Catalog resta sana?
2. Kill del Pod su Inventory a metà prenotazione: l'idempotenza più il modello di prenotazione impediscono l'oversell o il doppio addebito?
3. Disruption a livello di nodo / AZ: i PDB e il topology spread mantengono disponibile il percorso critico?
4. Partizione di rete / disruption di Kafka: la pipeline asincrona si riprende e converge una volta che la partizione si risana? Qualcosa va perso o duplicato?
5. Pod-failover del primario CloudNativePG: qual è l'impatto osservato sul checkout, e il sistema si riprende entro l'RTO dichiarato?

Ogni esperimento produce: l'ipotesi, la definizione dello stato stazionario, cosa avete osservato nelle vostre dashboard, se l'ipotesi ha retto, e cosa avete cambiato in caso contrario.

## Policy sul coding agentico

Gli strumenti di coding agentico sono permessi e incoraggiati per questo progetto, secondo tre regole che rispecchiano il modo in cui gli attori non umani sono trattati ovunque nel corso.

6. Potete usare gli agenti per produrre artefatti (scaffolding dei servizi, template Helm, dashboard, manifest, test harness). Non potete delegare i giudizi: la decomposizione dei servizi, il modello di consistenza, le definizioni degli SLO, la mappatura delle modalità di fallimento, le ipotesi di caos e il postmortem devono essere redatti e posseduti dal team.
7. L'agente è un attore nel vostro sistema, e dovete ragionare sul suo blast radius (raggio d'impatto). Se un agente di coding può aprire pull request contro il repository di configurazione che Argo CD riconcilia, allora è dentro il vostro control loop. Documentate: quali credenziali/permessi ha (privilegio minimo), se le sue modifiche richiedono revisione umana o policy-as-code prima del merge, e qual è il caso

peggiore se propone una modifica errata. È richiesto un breve paragrafo di threat-model.

8. Dovete verificare, e dovete mostrare dove l'agente ha sbagliato. Mantenete un breve docs/agent-log.md che registri almeno tre casi concreti in cui un artefatto prodotto da un agente era errato, non sicuro o sottilmente sbagliato (es. una probe di liveness che controlla una dipendenza downstream, un alert basato sulla causa, un ServiceAccount con permessi eccessivi, un handler non idempotente) e come il vostro team lo ha individuato e corretto. Questo viene valutato. Trovare gli errori dell'agente è l'abilità centrale che il capstone certifica.

Non sarete penalizzati per aver usato gli agenti in modo intensivo. Sarete penalizzati se nessuno nel team sa spiegare, difendere e operare ciò che è stato costruito.

## Modello di repository e delivery

Proseguite con il modello a due repository del Lab03/Lab04:

- Repository dell'applicazione: sorgente dei cinque servizi, test, workflow CI, justfile.
- Repository di configurazione: chart/manifest Helm per l'applicazione e per l'/le Application di Argo CD; bootstrap della piattaforma; definizioni di SLO/alert; documentazione del team.

I componenti della piattaforma installati una sola volta per cluster (Traefik, cert-manager, CloudNativePG, Strimzi, Sealed Secrets, Argo CD, kube-prometheus-stack, Chaos Mesh) sono infrastruttura condivisa, non workload per-app. I segreti vivono in Git solo come SealedSecrets (Lab04). La CI aggiorna lo stato desiderato; Argo CD distribuisce.

## Definition of Done

Il vostro team redige e versiona la propria DoD in `docs/capstone-dod.md`, ma deve coprire almeno i seguenti punti. Mantenete ogni voce breve e operativa.

- Progettazione & asincronia (Pilastro A)
- Consistenza (Pilastro B)
- Resilienza (Pilastro C)
- Delivery, osservabilità, prova (Pilastro D)
- Esperimenti di caos
- Coding agentico

## Ruoli del team suggeriti (5 persone, a rotazione, non createvi silos)

I ruoli coordinano la titolarità; tutti devono comprendere l'intero money path dall'inizio alla fine.

- Domain & async owner: confini dei servizi, pipeline degli ordini basata su coroutine/Flow, semantica di arresto.
- Data & consistency owner: CloudNativePG, modello di consistenza dell'inventario, idempotenza.
- Resilience owner: circuit breaker, bulkhead, probe, PDB, HPA, esperimenti Chaos Mesh.
- Delivery owner: GitOps, Argo CD, canary/blue-green, wiring di Strimzi/Kafka.
- Observability & verification owner: SLO, dashboard, alert, tracing e custode del log di verifica dell'agente.

Come in ogni lab: i ruoli servono per il coordinamento, non per costruire silos di conoscenza.

## Deliverable

9. I due repository (applicazione più configurazione), con una history che mostri una delivery guidata da GitOps.
10. docs/ contenente: la DoD, la giustificazione di progettazione/consistenza, le definizioni degli SLO, i report degli esperimenti di caos, il postmortem, il paragrafo di threat-model dell'agente, e agent-log.md.
11. Una demo registrata di circa 5 minuti (link committato nel repo) che mostri: il sistema in esecuzione, le dashboard che rispondono a domande operative, un deployment canary e uno blue/green, e almeno un alert che scatta sotto un fallimento iniettato.

Una volta che il sistema è pronto, contattateci per fissare una presentazione dal vivo. Presenterete il vostro sistema e risponderete alle domande tecniche relative al materiale del corso (esame orale).

## Valutazione del progetto

Il peso della valutazione segue la difficoltà che l'era agentica espone: il giudizio sopra la produzione.

- Resilienza dimostrata sotto fallimento (esperimenti di caos).
- Correttezza di consistenza & idempotenza sul money path.
- Osservabilità che guida realmente le decisioni (non volume di dashboard).
- Disciplina di delivery (GitOps, progressive delivery, nessuna credenziale del cluster nella CI).
- Correttezza dell'asincronia (cancellazione/arresto), e il ragionamento sui costi che vi sta dietro.
- Verifica & titolarità: il log degli errori dell'agente, il postmortem, e la vostra capacità di difendere il sistema che avete consegnato.

Un sistema scritto in gran parte dall'agente può comunque ottenere il massimo dei voti se il vostro team lo ha compreso, strumentato, rotto di proposito e riportato in funzione.

## Nota finale

Questo capstone chiude deliberatamente il cerchio che il corso ha tracciato fin dalla sua prima slide: dichiara l'intento, riconcilia in continuazione, assumi il fallimento, osserva tutto e fatti carico del risultato. Gli strumenti continueranno a cambiare, agenti inclusi. Il giudizio no. Quel giudizio è ciò che siete qui a dimostrare.